

**CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY**

COLLEGE OF COMPUTER STUDIES

Software Requirements Specifications
for
RESQMESH

Change History

Version	Date	Description	Author(s)
1.0	May 5, 2026	Initial release of SRS document	Alido, Libron, Plando, Salvame, Villarazo

Table of Contents

Change History	2
Table of Contents	3
1. Introduction	4
1.1. Purpose	4
1.2. Scope	4
1.3. Definitions, Acronyms and Abbreviations	4
1.4. References	4
2. Overall Description	5
2.1. Product perspective	5
2.2. User characteristics	5
2.4. Constraints	5
2.5. Assumptions and dependencies	6
3. Specific Requirements	7
3.1. External interface requirements	7
3.1.1. <i>Hardware interfaces</i>	7
3.1.2. <i>Software interfaces</i>	7
3.1.3. <i>Communications interfaces</i>	7
3.2. Functional requirements	7
<i>Module 1</i>	7
<i>Module 2</i>	8
3.4. Non-functional requirements	8
<i>Performance</i>	8
<i>Security</i>	8
<i>Reliability</i>	8

1. Introduction

1.1. Purpose

This document is the Software Requirements Specification (SRS) for ResQMesh, a decentralized emergency messaging system designed to facilitate communication during severe natural disasters and infrastructure collapses. The system addresses the critical inability of affected individuals to coordinate with first responders when centralized cellular towers and internet service providers fail due to power outages or structural damage.

This SRS is intended for the development team, software engineering advisors, and project stakeholders. It defines the complete functional and non-functional requirements that will govern the development of the ResQMesh mobile application, serving as the basis for the System Design Document (SDD) and subsequent minimum viable product (MVP) testing.

1.2. Scope

ResQMesh is an Android-based mobile platform that enables offline communication without cellular or internet connectivity. Utilizing Bluetooth Core Specification 5.4 and WiFi Direct, the system establishes a highly resilient, opportunistic mesh network directly between ubiquitous mobile devices.

From its localized deployment, the system performs the following core functions:

- **Offline Node Discovery:** Automated, battery-efficient background scanning to detect nearby compatible devices and establish secure peer-to-peer (P2P) sockets.
- **Decentralized Messaging:** Direct device-to-device private message transmission utilizing an entirely localized transport layer.
- **Data Hopping (Store-Carry-Forward):** An automated multi-hop relay mechanism where intermediate devices cache encrypted message payloads and physically transport them to out-of-range recipients as users move through the disaster zone.
- **Emergency Public Broadcasting:** A specialized flooding protocol designed to rapidly saturate the local network with high-priority SOS alerts.
- **End-to-End Encryption:** Strict cryptographic enforcement to ensure that intermediate relay nodes cannot intercept or decipher private communications.

The system is intended for civilians and emergency responders in disaster-stricken areas. It operates entirely off-grid and transmits zero user data to external cloud servers, operating strictly within a self-contained device-to-device ecosystem.

1.3. Definitions, Acronyms and Abbreviations

Term / Acronym	Definition
BLE PAwR	Bluetooth Low Energy Periodic Advertising with Responses. A Bluetooth 5.4 standard allowing bidirectional, connectionless communication in synchronized subevents, enabling communication with thousands of nodes simultaneously.
DTN	Delay-Tolerant Network. A network architecture operating under the assumption that an end-to-end routing path does not exist, relying instead on asynchronous message buffering and physical node mobility.
Store-Carry-Forward	A routing paradigm where a device receives a data packet, buffers it in local memory, physically transports it, and forwards it when a new node is encountered.
Mesh Network	A local network topology in which infrastructure nodes connect directly, dynamically, and non-hierarchically to as many other nodes as possible.
Data Hopping	The process of transmitting messages across a sequence of intermediate relay devices to reach a destination beyond the sender's direct physical range.
P2P	Peer-to-Peer - a decentralized communications model in which each party has the same capabilities and either party can initiate a communication session.
WiFi Direct	A Wi-Fi standard enabling devices to easily connect with each other without requiring a traditional wireless access point or router.
E2E	End-to-End Encryption - a secure communication standard preventing third parties, including relay nodes, from accessing data while it's transferred from one end system to another.
BSSA	Broadcast Storm Suppression Algorithm. A collection of mathematical techniques designed to prevent exponential packet duplication in wireless mesh networks by throttling redundant transmissions.
RSSI	Received Signal Strength Indicator. An estimated measurement of the power level of a radio frequency client signal, utilized inversely to estimate physical proximity.

1.4. References

- A. Vahdat and D. Becker, "Epidemic routing for partially connected ad hoc networks," Duke University, Technical Report, 2000.
- Bluetooth Special Interest Group, "Bluetooth core specification version 5.4," 2023. [Online]. Available: <https://www.bluetooth.com/specifications/specs/core-specification-5-4/>
- Wi-Fi Alliance, "Wi-Fi peer-to-peer (Wi-Fi Direct) technical specification," 2022. [Online]. Available: <https://www.wi-fi.org/discover-wi-fi/wi-fi-direct>
- K. Fall, "A delay-tolerant network architecture for challenged internets," in *Proceedings of the ACM SIGCOMM Conference*, 2003, pp. 27–34.
- S. Capkun, M. Hamdi, and J. P. Hubaux, "GPS-free positioning in mobile ad hoc networks," *Cluster Computing*, vol. 5, no. 2, pp. 157–167, 2002.

2. Overall Description

2.1. Product perspective

ResQMesh is an autonomous, self-contained Android application operating entirely independent of traditional telecommunications infrastructure. The system does not interface with cloud-based backends, web servers, or external databases. Instead, every instance of the application functions concurrently as a client interface, a cryptographic key generator, and an opportunistic network router. The foundational architecture relies on the Android Room persistent library, an abstraction layer over SQLite, which acts as the core DTN buffer. When a device loses connection with the mesh, data remains cryptographically sealed in this local database until the device's hardware radios detect a new, optimal forwarding opportunity.

2.2. User characteristics

- **Civilian Survivors:**
 - Requires an intuitive, low-friction interface allowing them to quickly broadcast SOS signals, share their status, or send encrypted messages to specific family members. They possess no technical knowledge of networking protocols. The application must operate silently in the background, autonomously managing node discovery and message routing without requiring the user to actively monitor the screen
- **Emergency Responders:**
 - Search and rescue personnel require a highly technical command interface. Their application instances are authorized to aggregate localized public broadcasts, triangulate map-less spatial coordinates based on incoming RF signals, and override local civilian hop limits to inject high-priority evacuation directives into the mesh. Responder devices are assumed to possess external power banks, allowing them to operate hardware radios at maximum transmission power with continuous duty cycles, acting as primary network anchors.

2.4. Constraints

- **Physical Transmission Physics:** The system's operational range is governed by the physical limitations of commercial smartphone antennas. Bluetooth 5.4 advertising packets reliably penetrate urban environments (e.g., collapsed drywall) up to approximately 30 meters, while Wi-Fi Direct can sustain TCP sockets up to 100 meters, strictly dependent on line-of-sight and multipath fading.
- **Battery Consumption:** Maintaining continuous active node discovery demands careful background scheduling to avoid draining the battery of victims who have no access to electricity.
- **Data Privacy Compliance:** As intermediate civilian devices will host and route data for strangers, all payloads must be heavily encrypted. No personally identifiable data may be exposed to the relay nodes.
- **OS Restrictions:** Modern Android versions aggressively throttle background processes. The system must navigate Android Doze mode and background execution limits to maintain the mesh.

2.5. Assumptions and dependencies

- **Device Availability:** It is assumed that a significant portion of the affected population possesses Android smartphones with intact, functioning internal batteries.
- **Critical Mass/Density:** The store-carry-forward mechanism relies heavily on human mobility and device density. It is assumed there will be enough circulating nodes (rescuers, evacuees) to act as "message ferries" across the disaster zone.
- **Hardware Compatibility:** It is assumed that the WiFi Direct and Bluetooth Low Energy (BLE) APIs function consistently across different Android manufacturer skins (Samsung, Xiaomi, etc.).

3. Specific Requirements

3.1. External interface requirements

3.1.1. Hardware interfaces

ResQMesh directly manipulates the device's native radio frequency adapters. No external hardware peripherals are required.

Hardware Interface	Execution Parameters
Bluetooth Low Energy (BLE) Adapter	The system utilizes the BLE hardware in dual mode, acting concurrently as a peripheral (broadcasting advertising packets) and a central (scanning for beacons). The architecture specifically leverages the BLE 5.4 Periodic Advertising with Responses (PAWR) specification, establishing precise micro-second time slots for bidirectional communication without initiating power-heavy connection events.
Wi-Fi Hardware Adapter	Utilized for the Wi-Fi Peer-to-Peer (P2P) framework. The hardware is invoked dynamically to establish high-throughput links for transferring accumulated DTN database buffers or multi-megabyte cryptographic key sets.

3.1.2. Software interfaces

The application is deeply integrated into the Android OS framework and utilizes modern asynchronous programming models.

Software Interface	Execution Parameters
Android WorkManager API	Replaces deprecated implicit broadcast receivers (e.g., <code>CONNECTIVITY_ACTION</code>). Used to schedule deterministic, periodic background tasks that align with the operating system's Doze mode maintenance windows, ensuring battery-efficient discovery scanning.
Android Room Persistence Library	An abstraction layer over SQLite utilized as the core DTN storage mechanism. Stores serialized, encrypted message payloads, TTL metrics, and the historical routing tables required for optimal stopping theory calculations.
Android Context & Permissions	Interfaces with the OS permission controller to secure <code>ACCESS_FINE_LOCATION</code> , <code>BLUETOOTH_CONNECT</code> , <code>BLUETOOTH_SCAN</code> , and specifically <code>NEARBY_WIFI_DEVICES</code> (required for API 33+).

3.1.3. Communications interfaces

The system implements a proprietary, multi-layered transport architecture that functions independently of traditional internet protocols.

- **Discovery Bearer:** Initial handshakes, proximity detection, and public SOS beacons are transmitted over the Bluetooth Advertising Bearer. This involves embedding custom mesh Protocol Data Units (PDUs) directly into the 31-byte BLE advertising payload, utilizing the PAwR synchronization slots.
- **Data Transport Bearer:** For bulk data transfer, the system establishes a localized Wi-Fi Direct TCP socket. The devices negotiate a Group Owner (GO) intent based on current battery levels, spinning up a temporary DHCP server to assign local IP addresses strictly for the duration of the data sync.

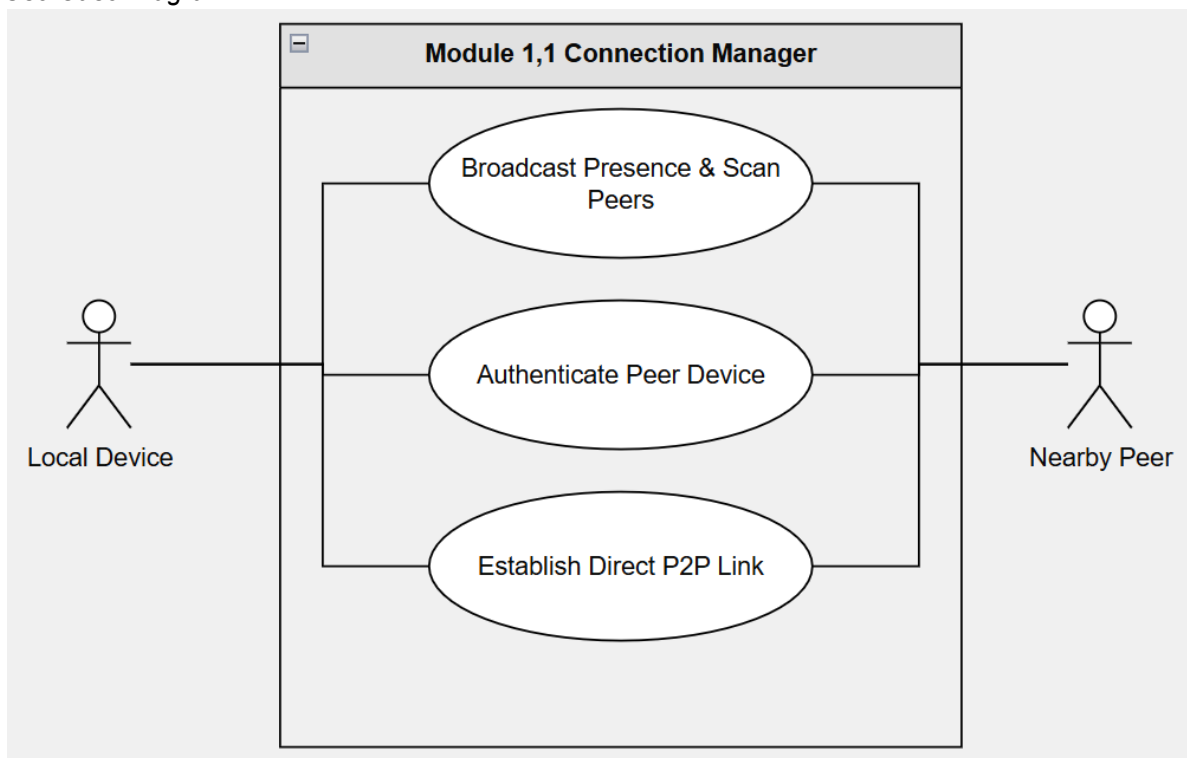
3.2. Functional requirements

Module 1: Offline Message Transmission Module

Automate direct offline message delivery between nearby mobile devices without dependence on cellular or internet connectivity by achieving $\geq 90\%$ successful message transmission within a 20-meter Bluetooth/WiFi Direct range.

1.1 Connection Manager

- Use Case Diagram



- Use Case Description

Use Case Name: Connection Manager

Primary Actor: Local Device

Secondary Actor: Nearby Peer

Description: Automates the wireless discovery and cryptographic pairing handshake between two offline devices to construct a local mesh topology.

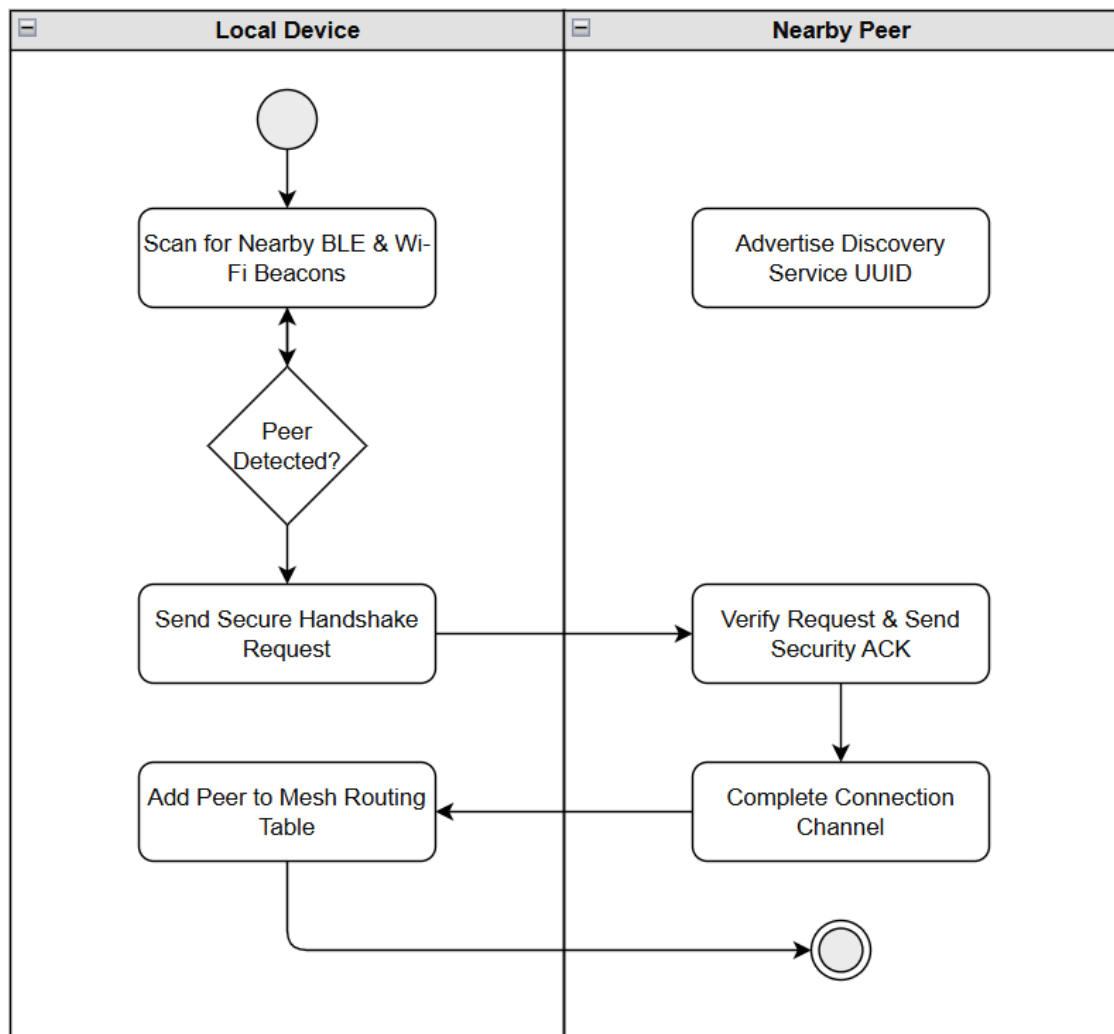
Pre-condition: Both devices have ResQMesh active, Bluetooth/WiFi radios enabled, and are within 20 meters of physical range.

Post-condition: A verified, secure direct P2P data tunnel is established within 15 seconds.

Main Flow:

1. Local Device activates its background discovery agent.
2. Nearby Peer continuously advertises its presence via BLE/WiFi Direct beacons.
3. Local Device intercepts the peer's beacon within the 20m range.
4. Local Device sends an automated connection request.
5. Nearby Peer automatically receives and validates the security credential handshake.
6. Connection is established, and both devices add each other to their routing tables.

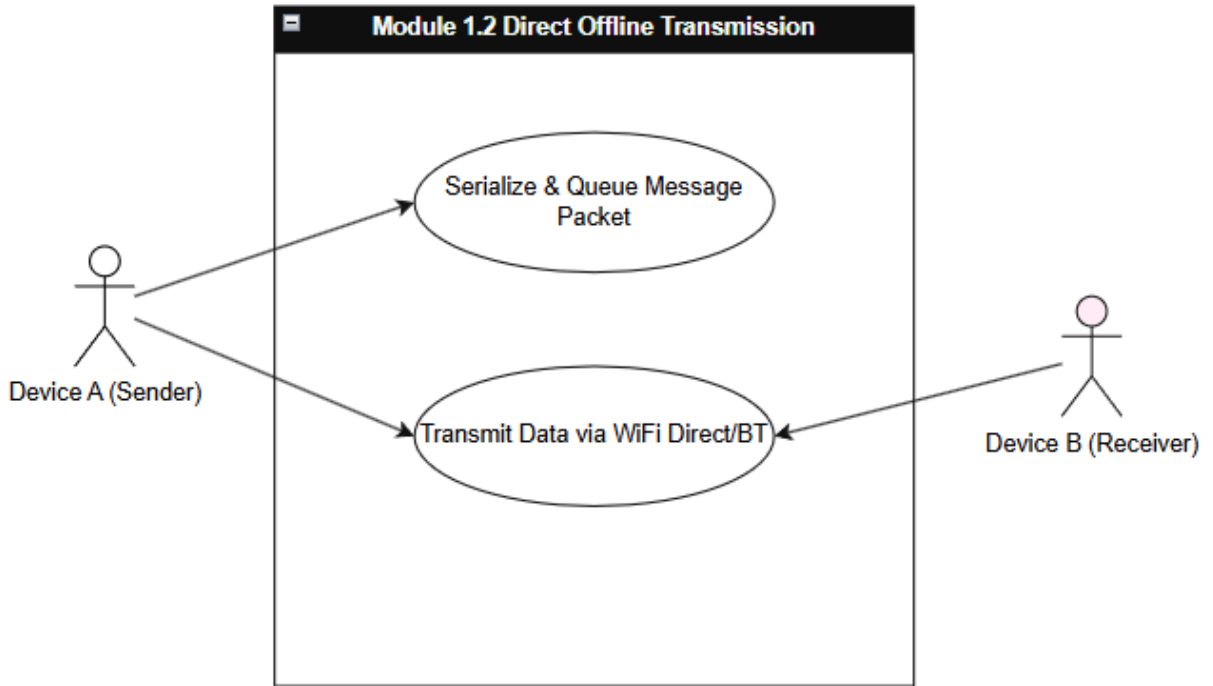
- Activity Diagram



- Wireframe

1.2 Direct Offline Transmission

- Use Case Diagram



- Use Case Description

Use Case Name: Direct Offline Transmission

Primary Actor: Device A (Sender)

Secondary Actor: Device B (Receiver)

Description: Automates the preparation and point-to-point transfer of encrypted data packets between two nearby devices using localized wireless protocols.

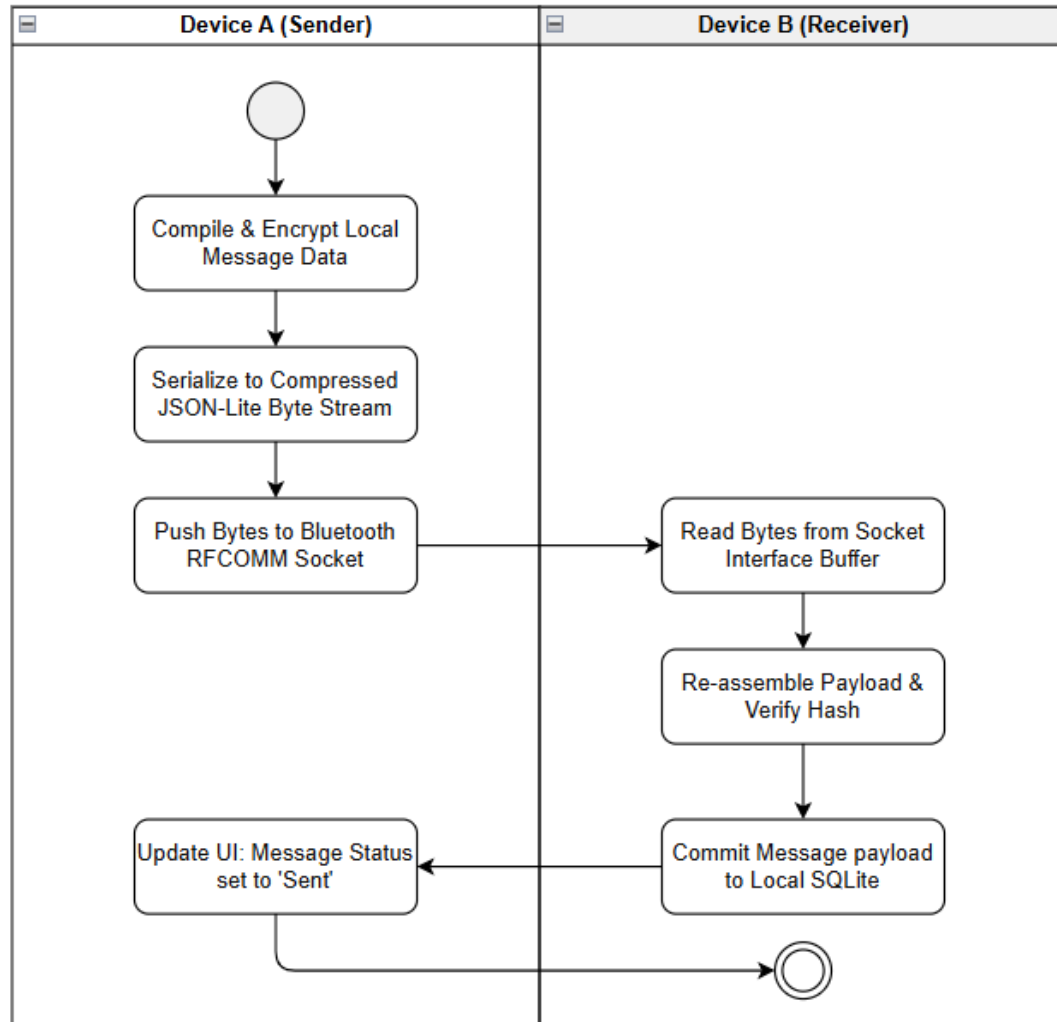
Pre-condition: A secure P2P link is established between Device A and Device B; Device A has a pending message in the transmission queue.

Post-condition: The data packet is successfully pushed to Device B's communication buffer within ≤ 5 seconds.

Main Flow:

1. Device A initiates the transmission process for a queued message.
2. System serializes the message data into optimized binary fragments (JSON-lite).
3. Device A opens a secure socket via Bluetooth or WiFi Direct.
4. Device A transmits the serialized packets directly to Device B.
5. Device B receives the bitstream via its active socket interface.

- Activity Diagram



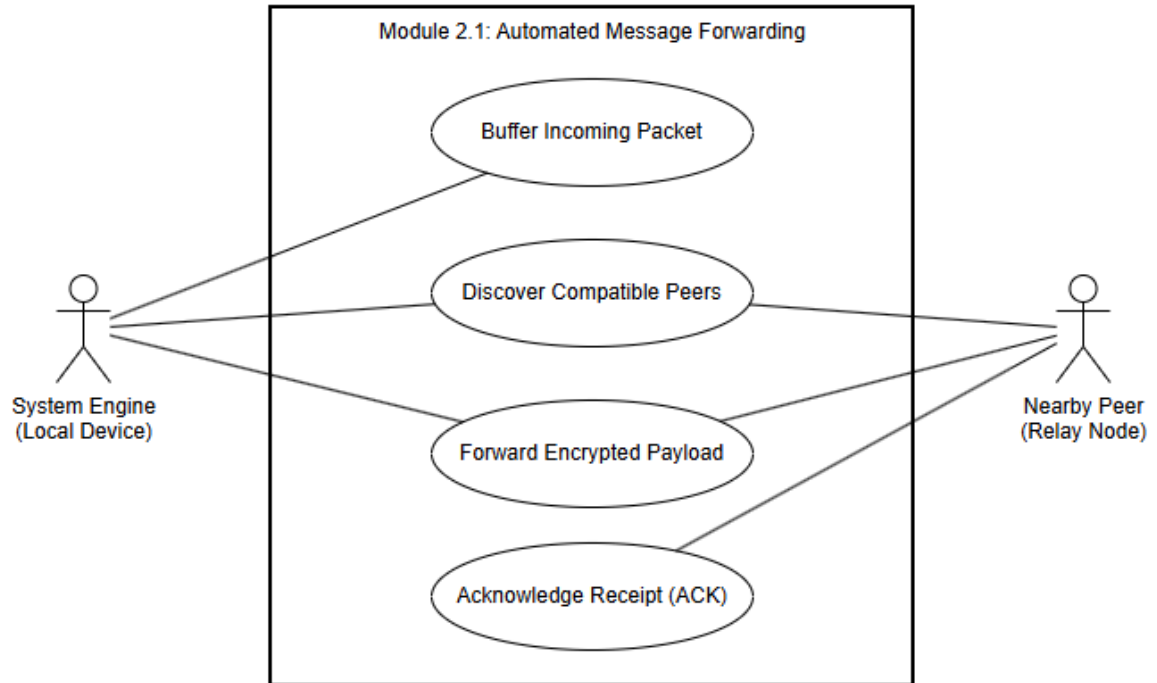
- Wireframe

Module 2: Network Relay and Broadcasting (Data Hopping) Module

Increase message reachability by extending transmission coverage through multi-device relay within 5 minutes.

2.1 Automate message forwarding to nearby devices with $\geq 85\%$ relay success rate

- Use Case Diagram



- Use Case Description

Use Case Name: Automated Message Forwarding (Store-Carry-Forward)

Primary Actor: System Engine (Local Device)

Secondary Actor: Nearby Peer (Relay Node)

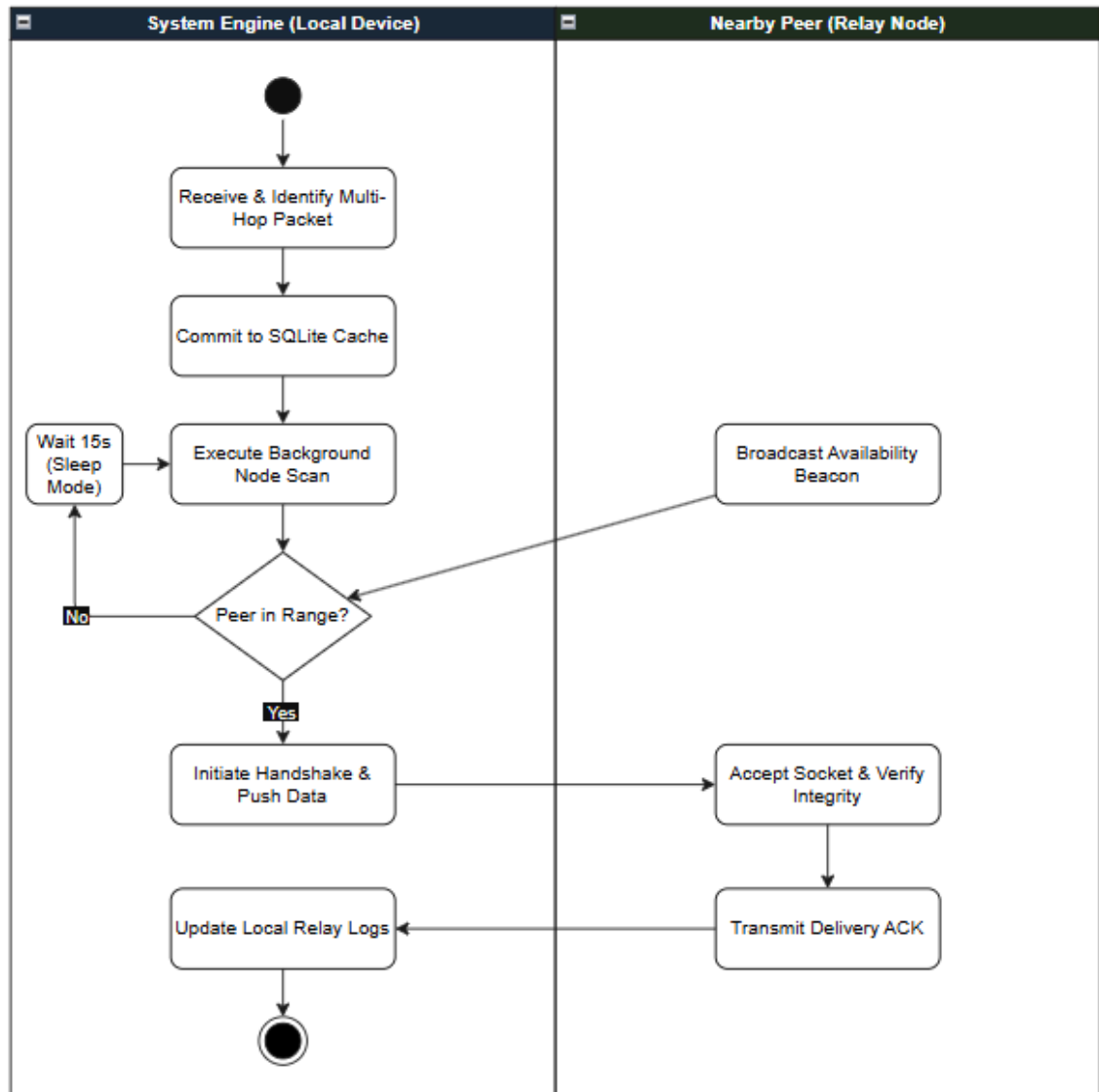
Description: The background service automates the core DTN (Delay-Tolerant Network) protocol by caching incoming packets in local memory and opportunistically pushing them to newly discovered peers within a 20-meter Bluetooth/WiFi Direct range, extending the network coverage without cellular dependence. **Pre-condition:** The Local Device has successfully received an encrypted message packet where it is not the intended final recipient. The device's Bluetooth/WiFi Direct adapters are active. **Post-condition:** The payload is successfully transmitted to a subsequent peer node, advancing the multi-hop delivery chain while remaining securely encrypted.

Main Flow:

1. The System Engine receives an incoming encrypted packet from the mesh.
2. The System Engine securely buffers the packet in the local SQLite database.
3. The background scanning service detects a compatible Nearby Peer broadcasting its availability.
4. The System Engine initiates and completes a secure handshake with the Nearby Peer.
5. The System Engine serializes and transmits the cached payload over the established P2P socket.
6. The Nearby Peer verifies packet integrity and returns a delivery acknowledgment (ACK).
7. The System Engine flags the specific packet as "Relayed" for that specific peer to prevent

localized echo loops.

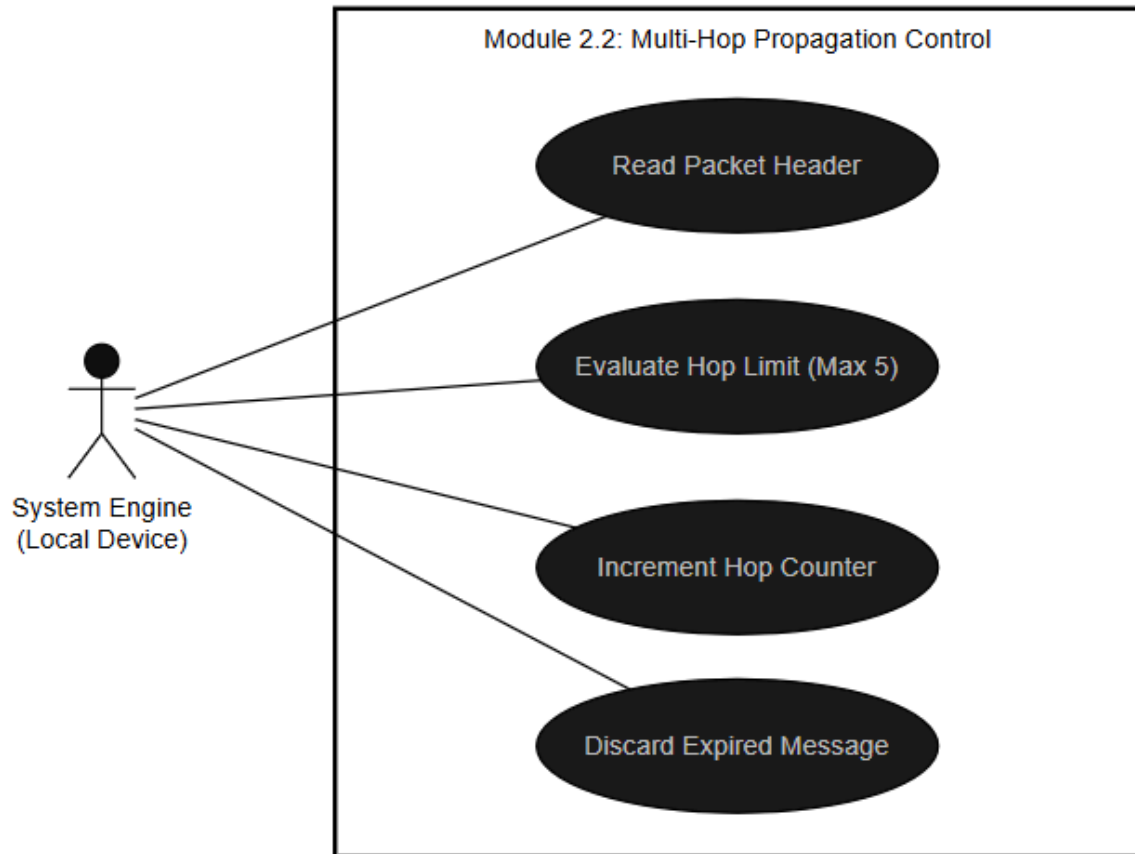
- Activity Diagram



- Wireframe

2.2 Multi-Hop Propagation Control

- Use Case Diagram



- *Use Case Description*

Use Case Name: Multi-Hop Propagation Control

Primary Actor: System Engine (Local Device)

Secondary Actor: None

Description: To prevent endless network loops and localized device memory overloads, the system evaluates the metadata of queued multi-hop messages. It strictly enforces a maximum threshold of 5 jumps before dropping the packet from the network.

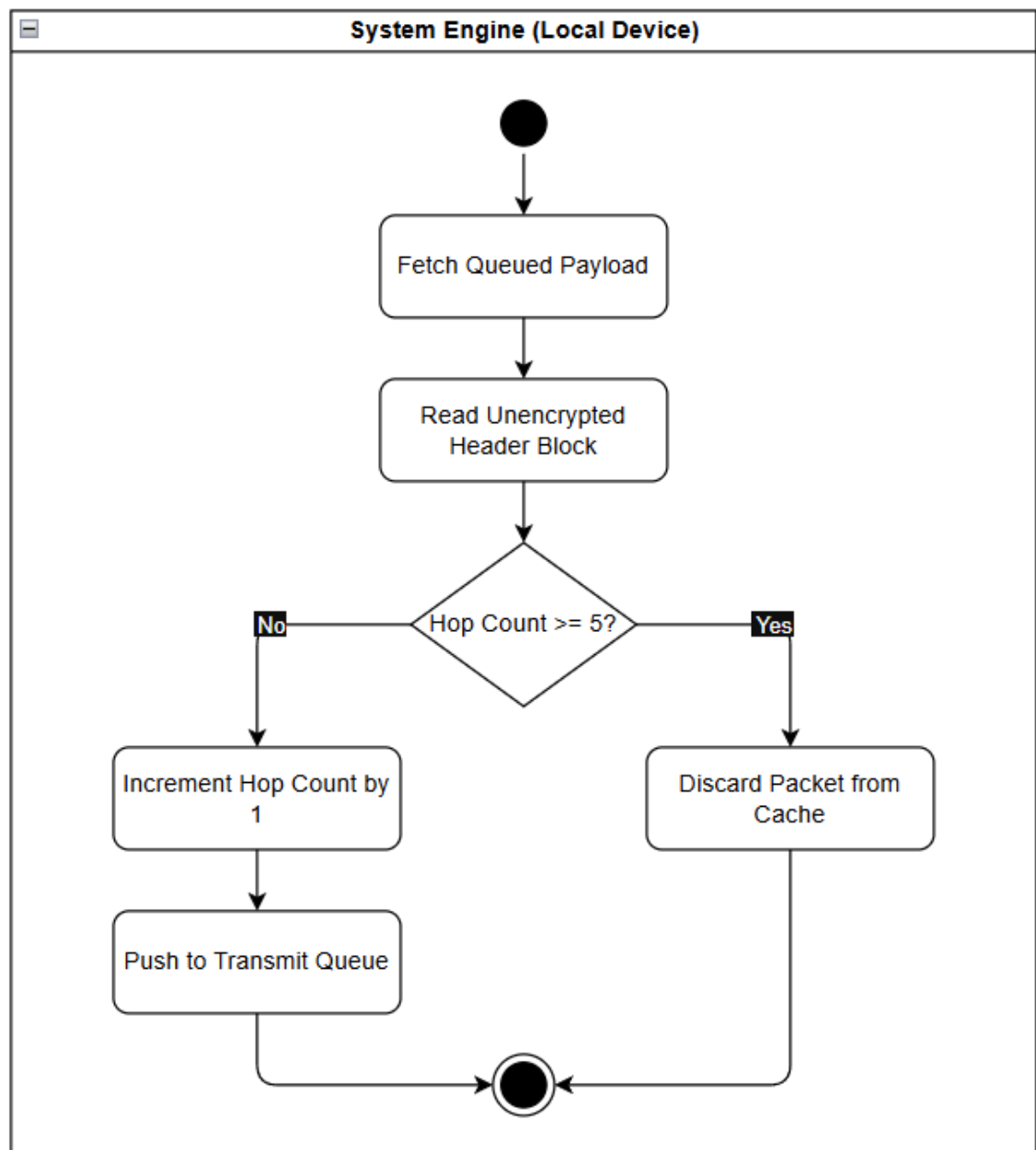
Pre-condition: A multi-hop packet has been received by the local device and placed into the local cache for potential relay evaluation.

Post-condition: The packet is either validated, its hop count incremented, and cleared for transmission, or it is permanently discarded to conserve bandwidth and network stability.

Main Flow:

1. The background System Engine selects a packet currently queued in the local SQLite database.
2. The system reads the unencrypted header block of the packet to extract the `current_hop_count` integer value.
3. The system executes a logic check: is `current_hop_count >= 5`?
4. If false (count is under the limit), the system increments the `current_hop_count` by 1.
5. The updated packet is pushed to the active transmission queue for peer-to-peer delivery.

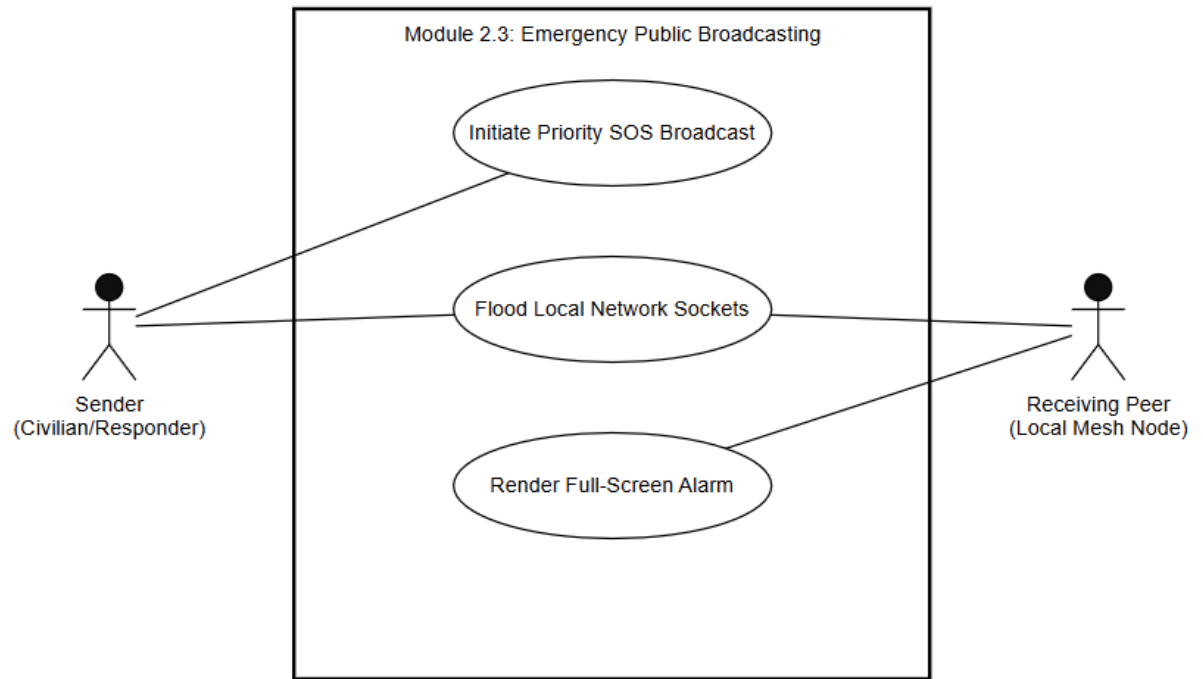
- *Activity Diagram*



- Wireframe

2.3 Emergency Public Broadcasting

- Use Case Diagram



- Use Case Description

Use Case Name: Emergency Public Broadcasting

Primary Actor: Sender (Civilian Survivor or Emergency Responder)

Secondary Actor: Receiving Peer (Nearby Mesh Node)

Description: In a critical emergency, a user initiates an unencrypted, high-priority SOS broadcast. The system bypasses standard optimized routing algorithms and deliberately floods the local network, pushing the payload to all currently connected peer devices simultaneously to achieve maximum local saturation rapidly.

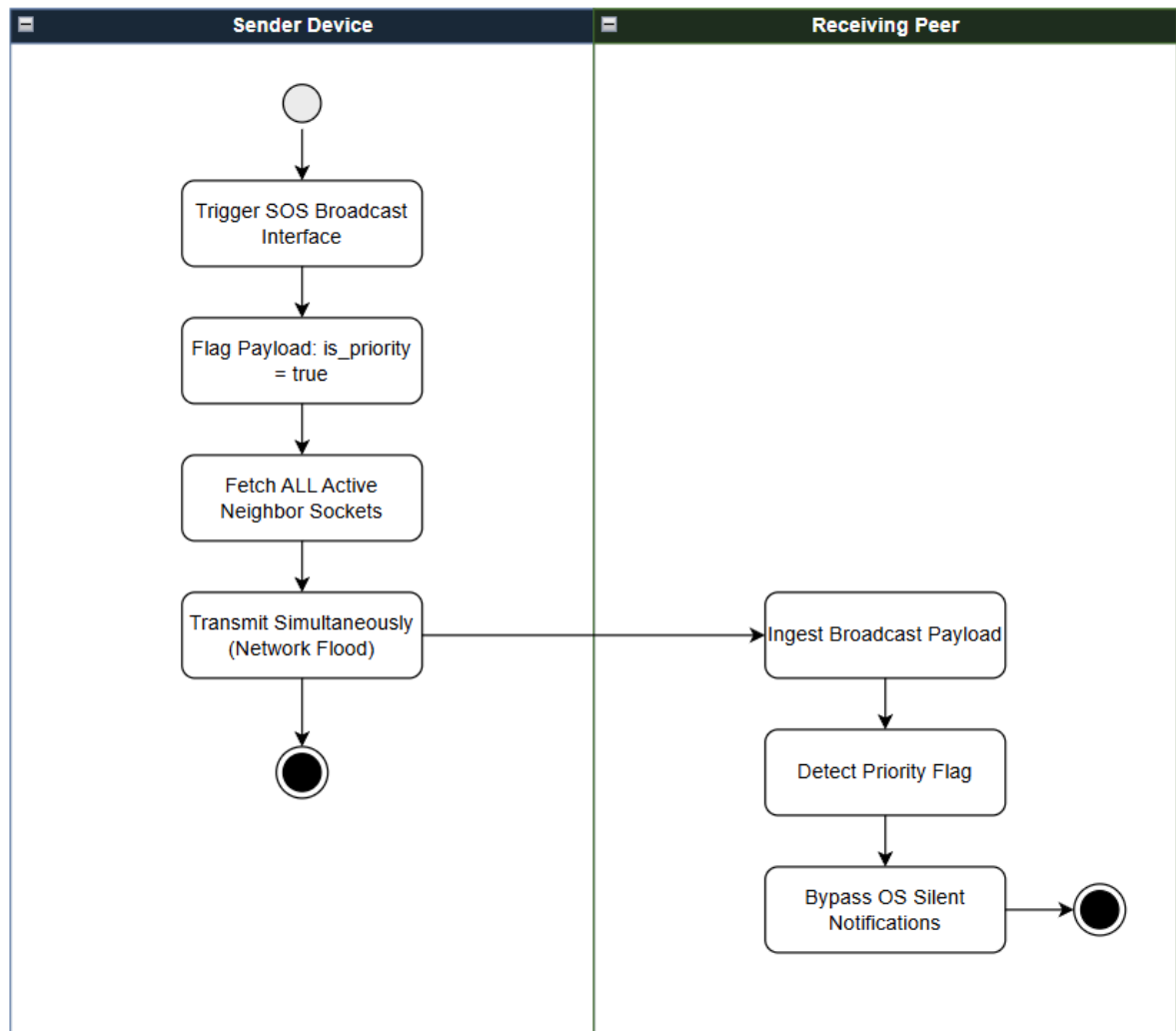
Pre-condition: The user has the ResQMesh application open and is connected to at least one neighboring node via Bluetooth/WiFi Direct.

Post-condition: The SOS message achieves a rapid local saturation (targeting $\geq 75\%$) and triggers immediate visual/audio alerts on the receiving devices.

Main Flow:

1. The Sender activates the "SOS Broadcast" feature on the application interface.
2. The System Engine packages the message and explicitly marks the metadata flag `is_priority_broadcast = true`.
3. The System Engine bypasses standard multi-hop path selection algorithms.
4. The System Engine queries the transport layer for all currently active hardware connections.
5. The system simultaneously pushes the broadcast payload to all active neighbor sockets.
6. The Receiving Peer ingests the payload and detects the priority flag.
7. The Receiving Peer's OS bypasses standard silent notification protocols and immediately renders a full-screen emergency UI alarm with audio (if volume settings permit).

- Activity Diagram



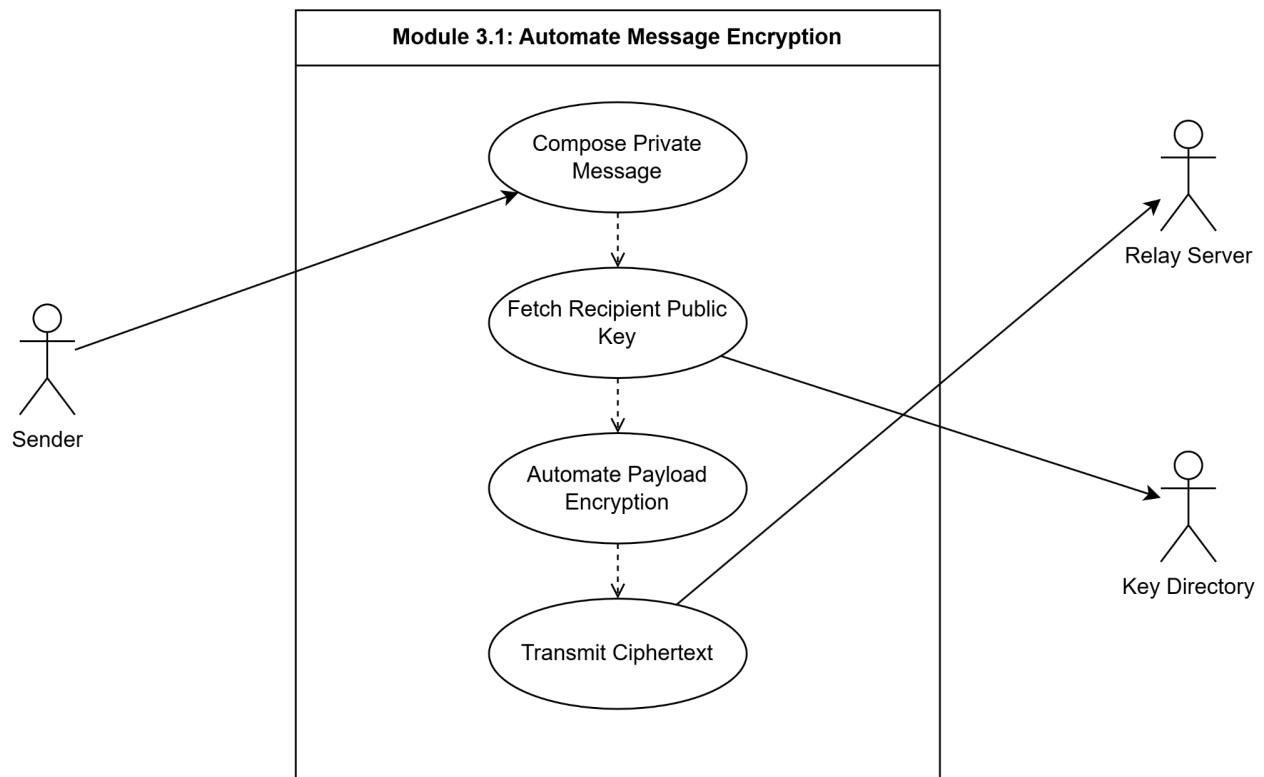
- Wireframe

Module 3: Privacy and Security Module

Enhance communication security by enforcing 100% end-to-end encryption to eliminate unauthorized message exposure.

3.1 Message Encryption and Transmission

- Use case Diagram



- Use case Description

Use Case Name: Automate Message Encryption

Primary Actor: Sender (User / Local Device)

Secondary Actor: Key Directory (Server), Relay Server (Network Node)

Description: This module encapsulates the processes required to compose a message and automate its end-to-end encryption before it ever leaves the user's local device. This guarantees a 100% secure transmission rate as only ciphertext is passed to the network layer.

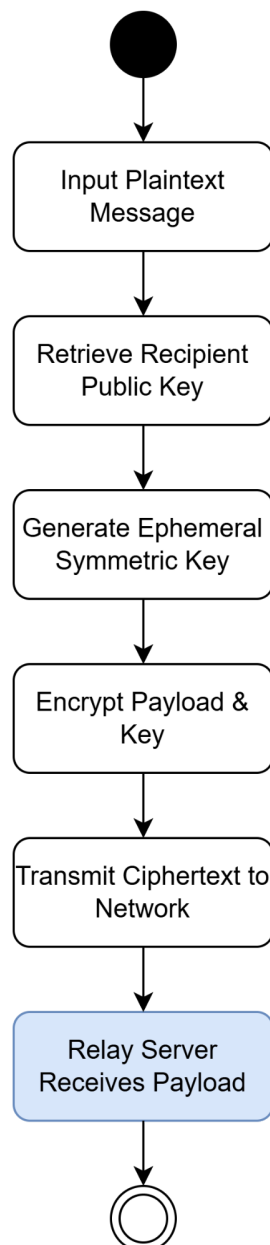
Pre-condition: The Sender is logged into the application, has an active internet connection, and has initiated a chat with a targeted recipient.

Post-condition: The plaintext message is destroyed from temporary local memory, and the securely encrypted ciphertext is successfully handed off to the network Relay Server.

Main Flow:

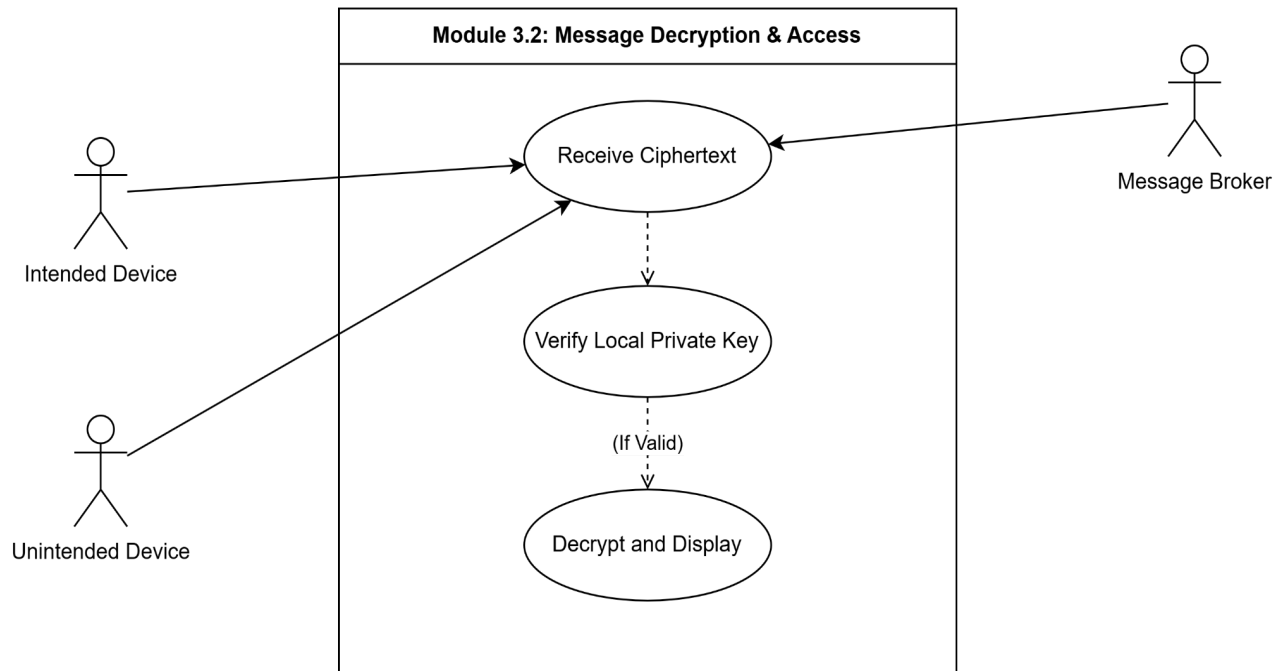
1. The Sender inputs a plaintext message into the application interface and clicks "Send."
2. The system automatically connects to the **Key Directory** to retrieve and cache the intended recipient's public cryptographic key.
3. The local system generates a one-time symmetric session key. It uses this session key to encrypt the plaintext message, and then uses the recipient's public key to encrypt the session key itself.
4. The local system bundles the encrypted payload and securely transmits it to the **Relay Server**, which will handle routing the packet through the network.

- *Activity Diagram*



3.2 Message Decryption and Access

- Use case Diagram



- Use case Description

Use Case Name: Secure Message Decryption and Access Control

Primary Actor: Intended Recipient Device

Secondary Actor: Message Broker (Sender of Payload), Unintended Device (Attacker)

Description: Ensures received ciphertext (delivered by the Message Broker) can only be decrypted by the exact device holding the matching private key, maintaining a 0% decryption success rate for unauthorized devices.

Pre-condition: The Message Broker has routed a ciphertext payload to a device.

Post-condition: The message is decrypted and displayed, or decryption fails and the payload is dropped securely.

Main Flow:

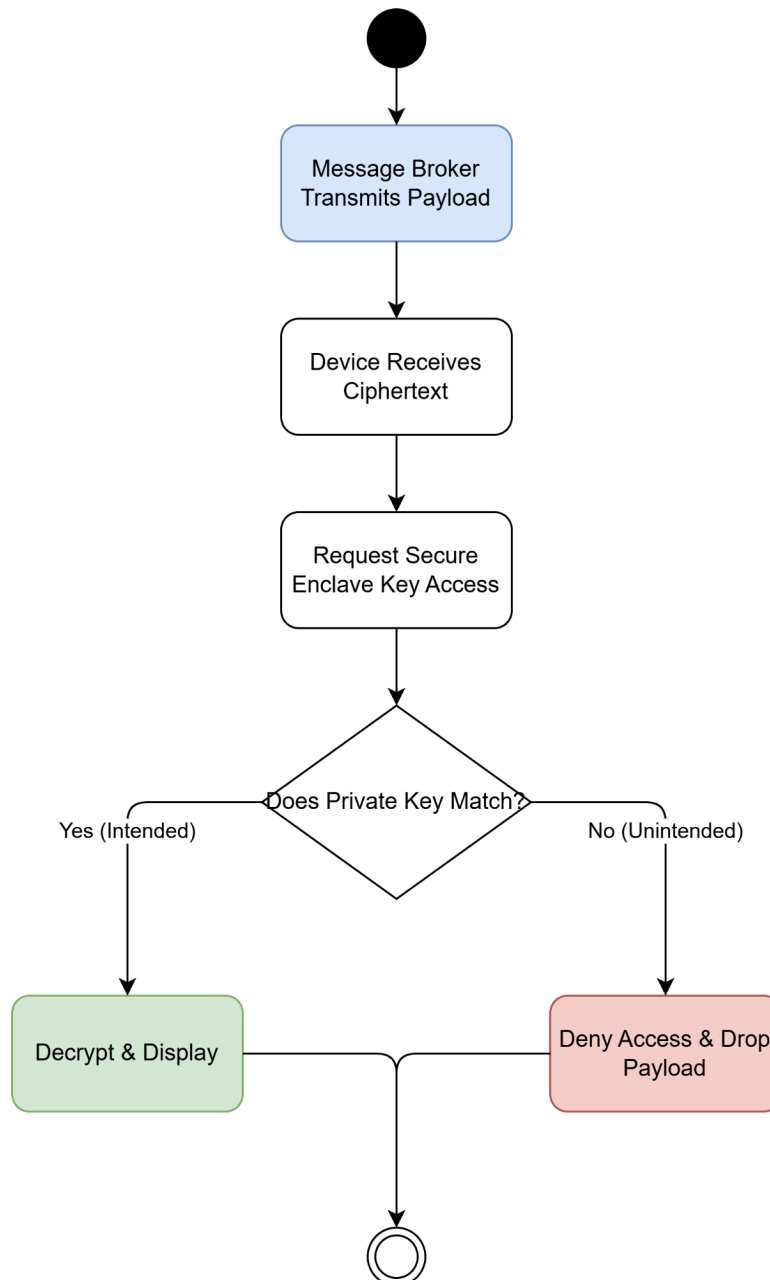
1. The Message Broker delivers the encrypted payload.
2. Device requests decryption from the local secure enclave.
3. The enclave uses the intended private key to decrypt the session key.
4. The session key decrypts the message payload.
5. Application displays plaintext to the user.

Alternative Scenario (Unauthorized Access)

1. An unintended device receives the payload from the Broker (due to interception or routing error).
2. Cryptographic verification fails because the private key does not match.

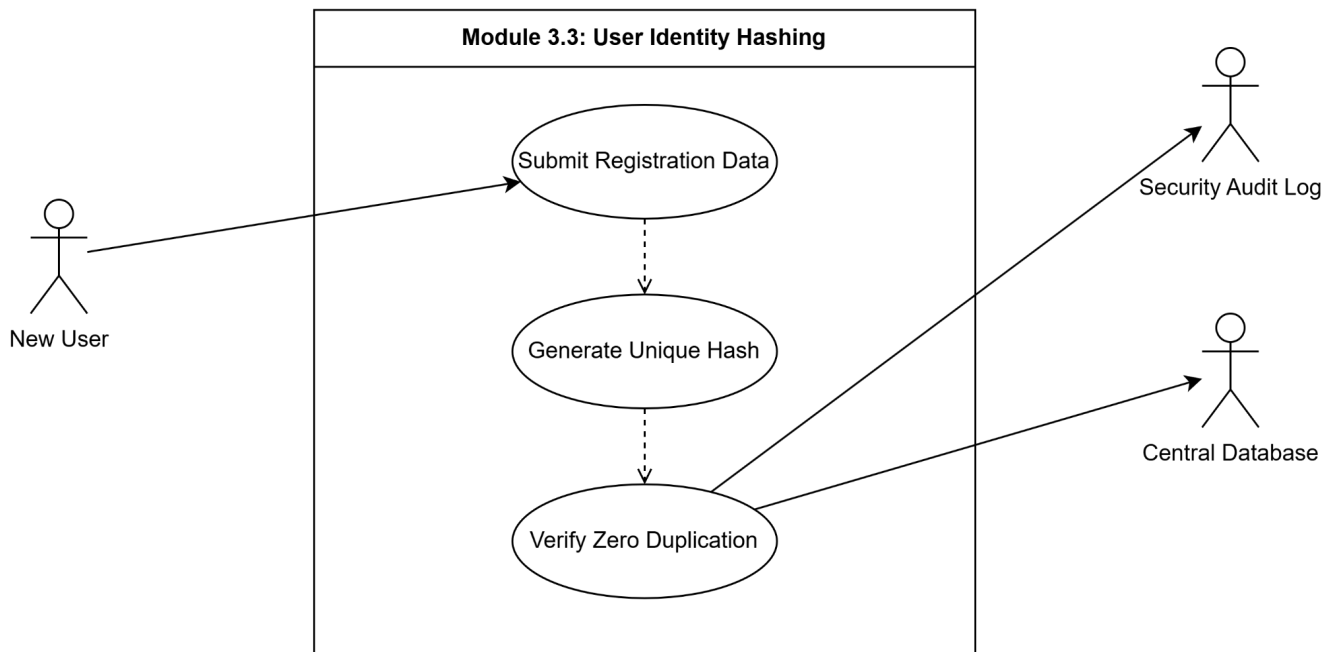
3. System immediately drops the payload. Decryption rate is 0%.

- Activity Diagram



3.3 User Identity Hashing and Deduplication

- Use Case Diagram



- Use case Description

Use Case Name: User Identity Hashing and Deduplication

Primary Actor: Registration System

Secondary Actor: Central User Database, Security Audit Log

Description: Automates the creation of a unique hashed identifier. Checks the Central Database to guarantee zero duplication, and writes events to a Security Audit Log to track potential abuse or duplicate attempts.

Pre-condition: A user submits identifying metadata for profile creation.

Post-condition: System either establishes a unique hashed ID, or denies the request and logs the duplicate attempt to the Security Audit Log.

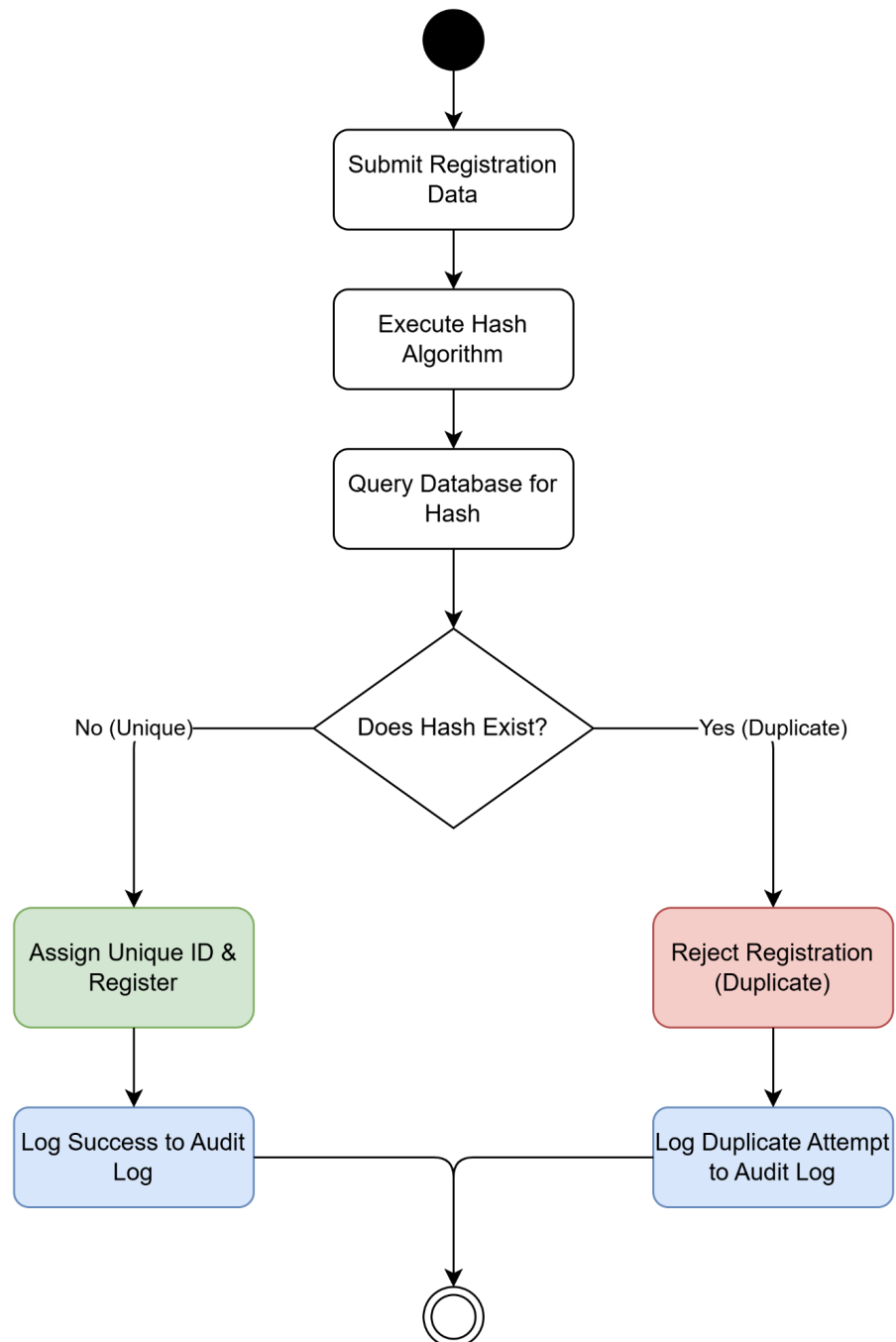
Main Flow:

1. System collects registration metadata.
2. System executes a one-way cryptographic hashing algorithm.
3. System queries Central Database using the hash.
4. Database returns "Not Found".
5. System assigns unique hash and creates the account.
6. Event is recorded in the Security Audit Log as a successful unique creation.

Alternative Scenario (Duplicate Detected)

1. In Step 4, Database returns "Exists".
2. System halts the registration to enforce zero duplication.
3. System flags the attempt and writes an alert to the Security Audit Log.

- Activity Diagram

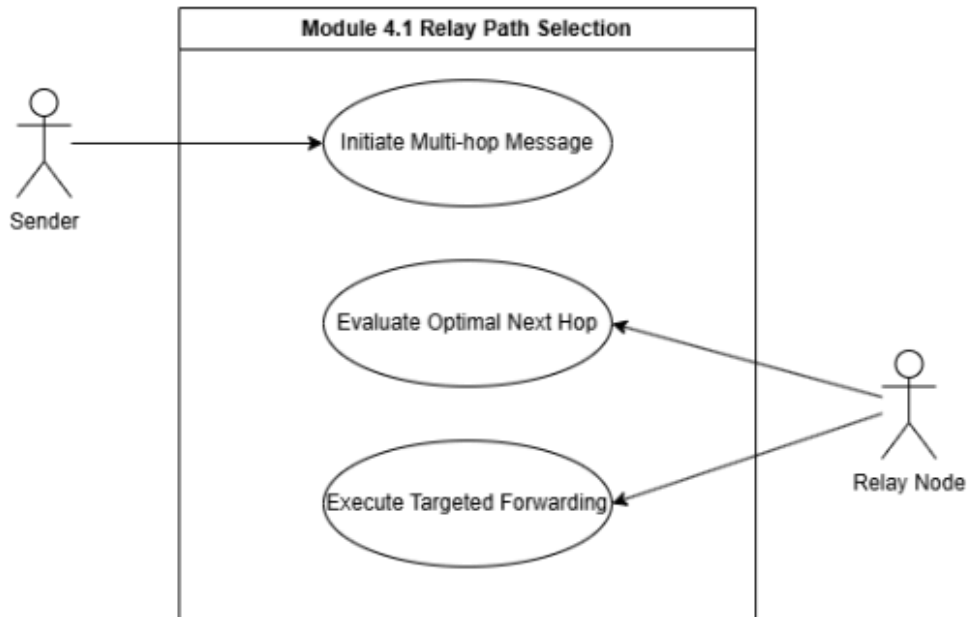


Module 4: Message Routing and Relay Verification Module

Improve indirect message delivery by optimizing message routing and relay verification to achieve $\geq 85\%$ successful multi-hop transmission accuracy.

4.1 Routing Messages

- Use Case Diagram



- Use Case Description

Use Case Name: Relay Path Selection

Primary Actor: **Relay Node** (Intermediate Mobile Device)

Secondary Actor: **Sender** (Original Message Initiator)

Description: The system evaluates the proximity and signal strength of all connected neighbors to forward the message to the single most optimal peer rather than broadcasting it to the entire local network.

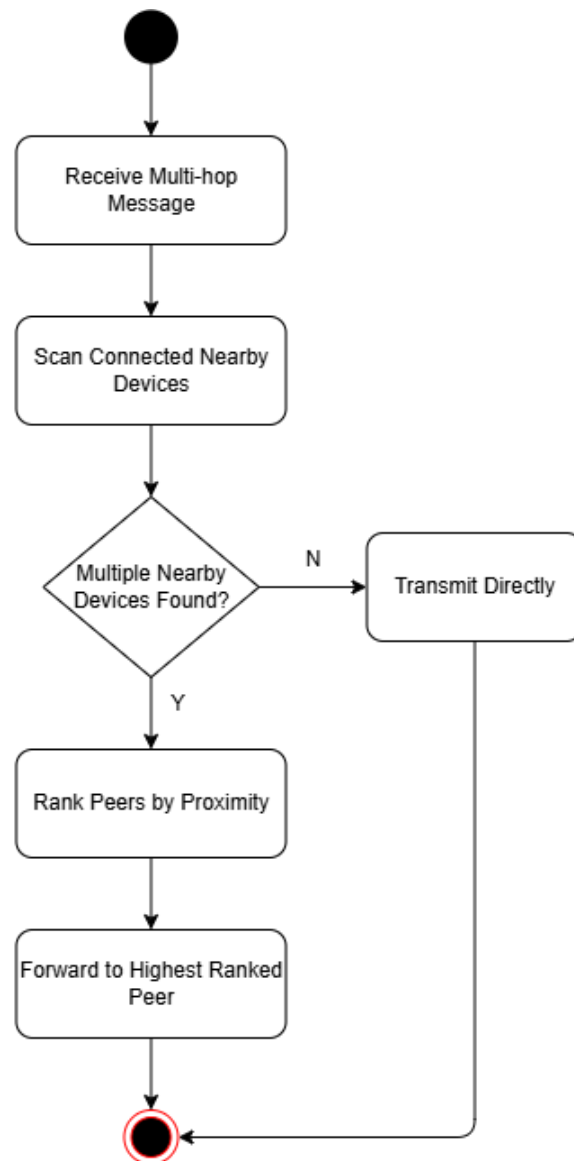
Pre-condition: The Sender has initiated a multi-hop message; the Relay Node is within range of at least one other peer.

Post-condition: The message is successfully forwarded to a targeted neighbor, minimizing overall network traffic.

Main Flow:

1. Sender initiates a message intended for a distant recipient
2. Relay Node receives the message and scans for active Bluetooth/WiFi Direct neighbors.
3. System ranks neighbors based on proximity.
4. System selects the highest-ranked neighbor and transmits the message.

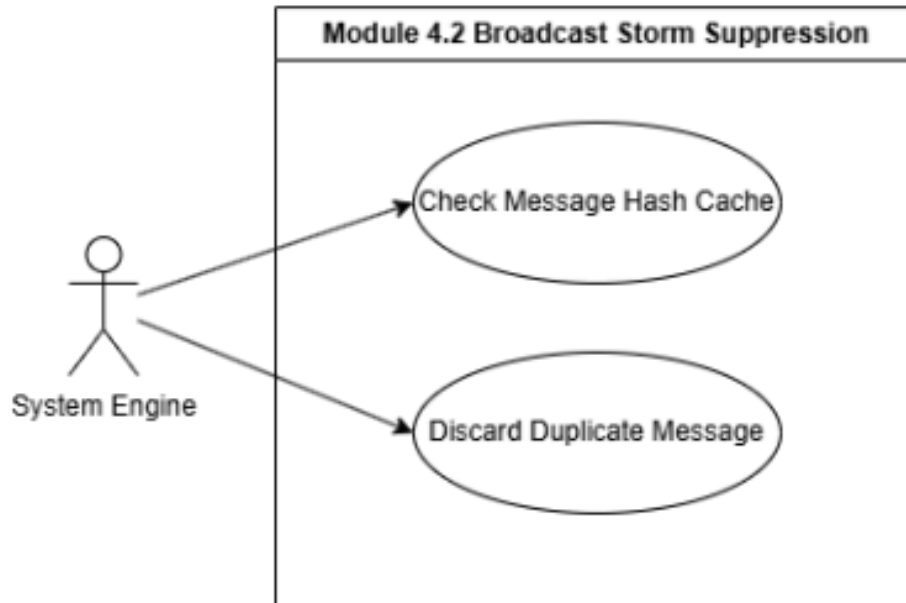
- Activity Diagram



- Wireframe

4.2 Broadcast Storm Suppression

- Use Case Diagram



- Use Case Description

Actor: Device Messaging Engine.

Description: The system checks the unique hashed identifier of an incoming message against a local "Relay History" cache. If the ID exists, the message is discarded to prevent infinite loops (echoes).

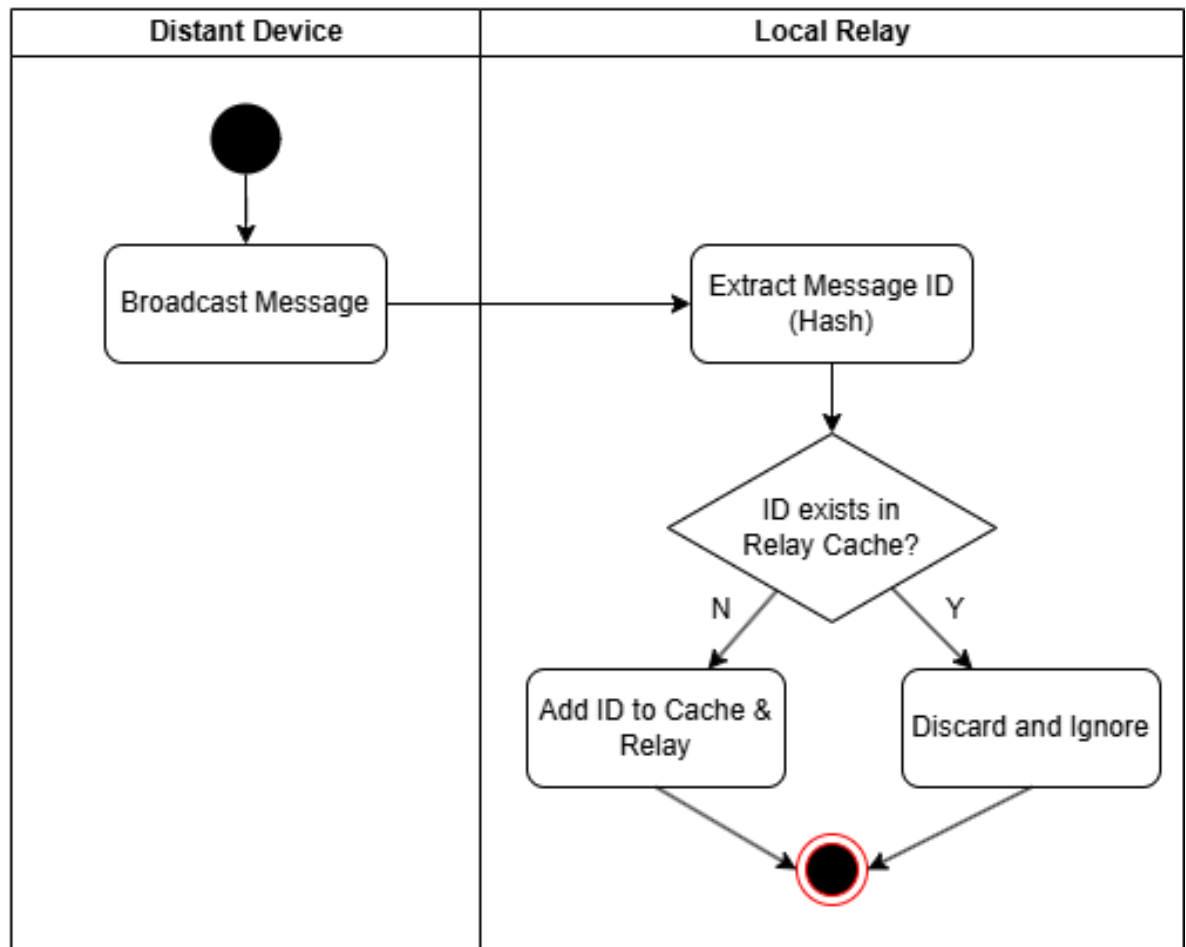
Pre-condition: Incoming broadcast message detected.

Post-condition: Redundant message suppressed (0 echoes).

Main Flow

1. Distant Device broadcasts a message to the mesh.
2. Local Node receives the packet and extracts the unique Message ID (Hash).
3. System compares the ID against its local Relay History cache.
4. If ID is found, the message is discarded immediately.
5. If ID is new, the message is added to the cache and processed for further relay.

- Activity Diagram

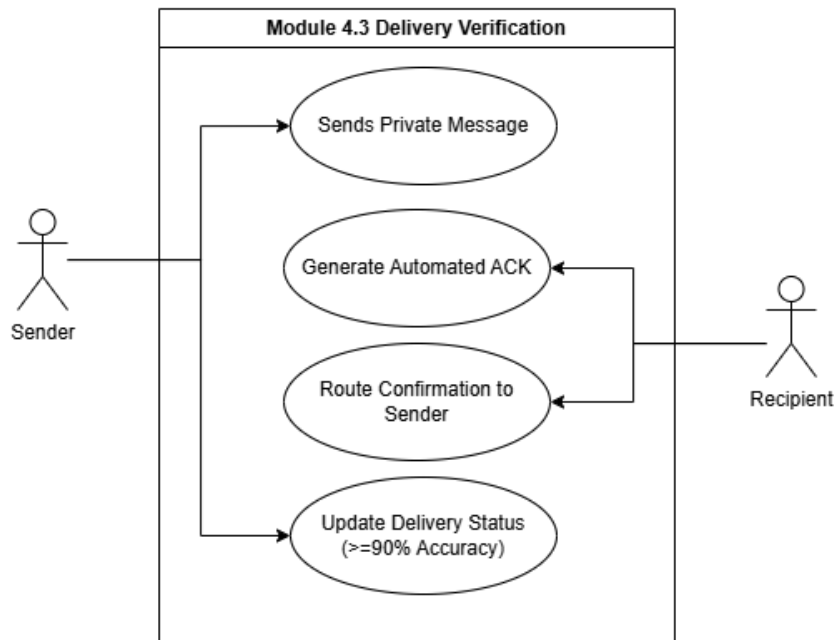


- Wireframe

...

4.3 Delivery Verification

- Use Case Diagram



- Use Case Description

Primary Actor: Recipient (Intended Target)

Secondary Actor: Sender (Original Message Source)

Description: Once a private message is successfully decrypted by the target, an automated Acknowledgement (ACK) packet is generated and routed back to the Sender to confirm delivery.

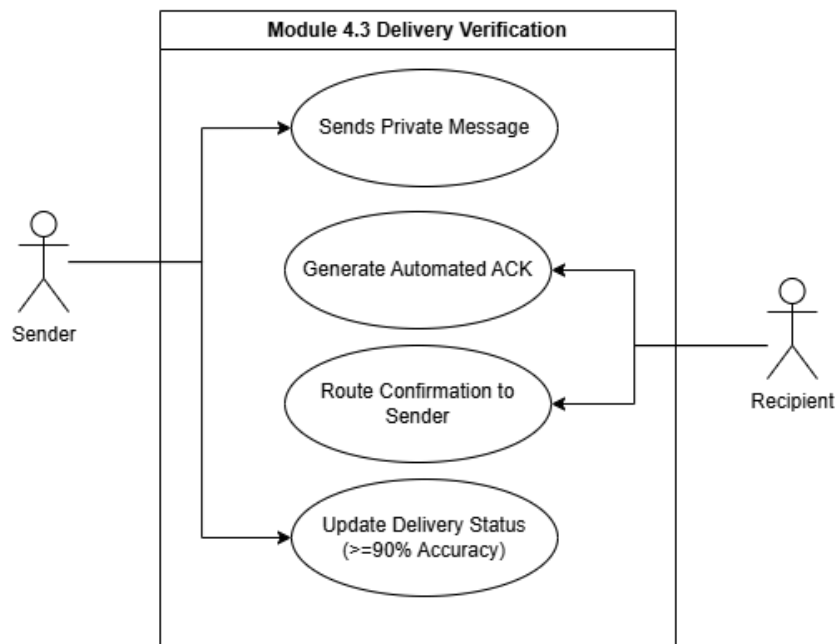
Pre-condition: Message has successfully reached the Recipient and passed end-to-end encryption verification.

Post-condition: Sender's UI updates to show a "Delivered" status with high accuracy.

Main Flow

1. Recipient device decrypts the incoming private message.
2. System automatically generates a signed ACK packet tied to the original Message ID.
3. ACK packet is transmitted back into the mesh using reverse-routing.
4. Sender device receives the ACK and updates the message status on the local interface.

- **Activity Diagram**



- **Wireframe**

3.4 Non-functional requirements

Performance

Discovery Speed: The system must automate nearby device detection and establish secure P2P connections within ≤ 15 seconds of entering physical range.

Latency: Direct device-to-device message transmission (1 hop) must execute with ≤ 5 seconds of latency.

Network Saturation: The emergency public broadcasting feature must achieve a $\geq 75\%$ local network saturation rate within a 5-minute window during testing scenarios.

Security

Zero PII Leakage: The application must strictly enforce 100% end-to-end encryption for all non-broadcast payloads, maintaining a guaranteed 0% decryption success rate for unintended recipient devices acting as intermediate relays.

Cryptographic Identity: The system must automate the assignment of unique, hashed identifiers to guarantee zero user duplication conflicts across the decentralized network, completely bypassing the need for a centralized authentication server.

Reliability

Relay Stability: The automated store-carry-forward relay module must maintain a $\geq 85\%$ successful relay rate under optimal hardware conditions.

Congestion Control: Message routing and path selection protocols must reduce redundant message forwarding (broadcast storms) to $\leq 10\%$ of total network traffic by ensuring previously relayed messages are echoed 0 times by the same device.

Hop Limits: Message propagation must be strictly limited to a maximum of 5 intermediate devices to maintain network stability and prevent local device memory overloads.